Solvers

> **Note**
>
> For a beginner-friendly guide focused on solving common types of equations, refer to Solve Equations.

```
>>> from sympy import *
>>> x, y, z = symbols('x y z')
>>> init_printing(use_unicode=True)
```

A Note about Equations

Recall from the gotchas section of this tutorial that symbolic equations in SymPy are not represented by `=` or `==`, but by `Eq`.

```
>>> Eq(x, y)
x = y
```

However, there is an even easier way. In SymPy, any expression not in an `Eq` is automatically assumed to equal 0 by the solving functions. Since $a = b$ if and only if $a - b = 0$, this means that instead of using `x == y`, you can just use `x - y`. For example

```
>>> solveset(Eq(x**2, 1), x)
{-1, 1}
>>> solveset(Eq(x**2 - 1, 0), x)
{-1, 1}
>>> solveset(x**2 - 1, x)
{-1, 1}
```

This is particularly useful if the equation you wish to solve is already equal to 0. Instead of typing `solveset(Eq(expr, 0), x)`, you can just use `solveset(expr, x)`.

Solving Equations Algebraically

The main function for solving algebraic equations is `solveset`. The syntax for `solveset` is `solveset(equation, variable=None, domain=S.Complexes)` Where `equations` may be in the form of `Eq` instances or expressions that are assumed to be equal to zero.

Please note that there is another function called `solve` which can also be used to solve

Skip to content

ıtax is `solve(equations, variables)` However, it is recommended to use

.

When solving a single equation, the output of `solveset` is a `FiniteSet` or an `Interval` or `ImageSet` of the solutions.

```
>>> solveset(x**2 - x, x)
{0, 1}
>>> solveset(x - x, x, domain=S.Reals)
ℝ
>>> solveset(sin(x) - 1, x, domain=S.Reals)
⎧        π │       ⎫
⎨2⋅n⋅π + ─ │ n ∈ ℤ⎬
⎩        2 │       ⎭
```

If there are no solutions, an `EmptySet` is returned and if it is not able to find solutions then a `ConditionSet` is returned.

```
>>> solveset(exp(x), x)     # No solution exists
∅
>>> solveset(cos(x) - x, x)  # Not able to find solution
{x │ x ∈ ℂ ∧ (-x + cos(x) = 0)}
```

In the `solveset` module, the linear system of equations is solved using `linsolve`. In future we would be able to use linsolve directly from `solveset`. Following is an example of the syntax of `linsolve`.

- List of Equations Form:

```
>>> linsolve([x + y + z - 1, x + y + 2*z - 3 ], (x, y, z))
{(-y - 1, y, 2)}
```

- Augmented Matrix Form:

```
>>> linsolve(Matrix(([1, 1, 1, 1], [1, 1, 2, 3])), (x, y, z))
{(-y - 1, y, 2)}
```

- A*x = b Form

```
>>> M = Matrix(((1, 1, 1, 1), (1, 1, 2, 3)))
>>> system = A, b = M[:, :-1], M[:, -1]
>>> linsolve(system, x, y, z)
{(-y - 1, y, 2)}
```

> **Note**
>
> The order of solution corresponds the order of given symbols.

Skip to content

In the `solveset` module, the non linear system of equations is solved using `nonlinsolve`. Following are examples of `nonlinsolve`.

1. When only real solution is present:

```
>>> a, b, c, d = symbols('a, b, c, d', real=True)
>>> nonlinsolve([a**2 + a, a - b], [a, b])
{(-1, -1), (0, 0)}
>>> nonlinsolve([x*y - 1, x - 2], x, y)
{(2, 1/2)}
```

2. When only complex solution is present:

```
>>> nonlinsolve([x**2 + 1, y**2 + 1], [x, y])
{(-ⅈ, -ⅈ), (-ⅈ, ⅈ), (ⅈ, -ⅈ), (ⅈ, ⅈ)}
```

3. When both real and complex solution are present:

```
>>> from sympy import sqrt
>>> system = [x**2 - 2*y**2 -2, x*y - 2]
>>> vars = [x, y]
>>> nonlinsolve(system, vars)
{(-2, -1), (2, 1), (-√2⋅ⅈ, √2⋅ⅈ), (√2⋅ⅈ, -√2⋅ⅈ)}
```

```
>>> system = [exp(x) - sin(y), 1/y - 3]
>>> nonlinsolve(system, vars)
{({2⋅n⋅ⅈ⋅π + log(sin(1/3)) │ n ∈ ℤ}, 1/3)}
```

4. When the system is positive-dimensional system (has infinitely many solutions):

```
>>> nonlinsolve([x*y, x*y - x], [x, y])
{(0, y)}
```

```
>>> system = [a**2 + a*c, a - b]
>>> nonlinsolve(system, [a, b])
{(0, 0), (-c, -c)}
```

Skip to content

Note

1. The order of solution corresponds the order of given symbols.

2. Currently `nonlinsolve` doesn't return solution in form of `LambertW` (if there is solution present in the form of `LambertW`).

`solve` can be used for such cases:

```
>>> solve([x**2 - y**2/exp(x)], [x, y], dict=True)
⎡⎧          ____⎫  ⎧          ____⎫⎤
⎢⎨         ╱  x ⎬  ⎨         ╱  x ⎬⎥
⎣⎩y: -x⋅╲╱  ℯ  ⎭, ⎩y: x⋅╲╱  ℯ  ⎭⎦
>>> solve(x**2 - y**2/exp(x), x, dict=True)
⎡⎧      ⎛-y ⎞⎫  ⎧    ⎛y⎞⎫⎤
⎢⎨x: 2⋅W⎜──⎟⎬, ⎨x: 2⋅W⎜─⎟⎬⎥
⎣⎩      ⎝ 2 ⎠⎭  ⎩    ⎝2⎠⎭⎦
```

3. Currently `nonlinsolve` is not properly capable of solving the system of equations having trigonometric functions.

`solve` can be used for such cases (but does not give all solution):

```
>>> solve([sin(x + y), cos(x - y)], [x, y])
⎡⎛-3⋅π  3⋅π⎞  ⎛-π  π⎞  ⎛π  3⋅π⎞  ⎛3⋅π  π⎞⎤
⎢⎜─────, ───⎟, ⎜──, ─⎟, ⎜─, ───⎟, ⎜───, ─⎟⎥
⎣⎝  4     4⎠  ⎝ 4  4⎠  ⎝4   4 ⎠  ⎝ 4  4⎠⎦
```

`solveset` reports each solution only once. To get the solutions of a polynomial including multiplicity use `roots`.

```
>>> solveset(x**3 - 6*x**2 + 9*x, x)
{0, 3}
>>> roots(x**3 - 6*x**2 + 9*x, x)
{0: 1, 3: 2}
```

The output `{0: 1, 3: 2}` of `roots` means that `0` is a root of multiplicity 1 and `3` is a root of multiplicity 2.

Note

Currently `solveset` is not capable of solving the following types of equations:

- Equations solvable by LambertW (Transcendental equation solver).

`solve` can be used for such cases:

Skip to content

```
exp(x) - 1, x )
```

Solving Differential Equations

To solve differential equations, use `dsolve`. First, create an undefined function by passing `cls=Function` to the `symbols` function.

```
>>> f, g = symbols('f g', cls=Function)
```

`f` and `g` are now undefined functions. We can call `f(x)`, and it will represent an unknown function.

```
>>> f(x)
f(x)
```

Derivatives of `f(x)` are unevaluated.

```
>>> f(x).diff(x)
d
──(f(x))
dx
```

(see the Derivatives section for more on derivatives).

To represent the differential equation $f''(x) - 2f'(x) + f(x) = \sin(x)$, we would thus use

```
>>> diffeq = Eq(f(x).diff(x, x) - 2*f(x).diff(x) + f(x), sin(x))
>>> diffeq
                     2
         d           d
f(x) - 2⋅──(f(x)) + ───(f(x)) = sin(x)
         dx           2
                    dx
```

To solve the ODE, pass it and the function to solve for to `dsolve`.

```
>>> dsolve(diffeq, f(x))
                      x   cos(x)
f(x) = (C₁ + C₂⋅x)⋅ℯ  + ──────
                          2
```

`dsolve` returns an instance of `Eq`. This is because, in general, solutions to differential equations cannot be solved explicitly for the function.

```
>>> dsolve(f(x).diff(x)*(1 - sin(f(x))) - 1, f(x))
x - f(x) - cos(f(x)) = C₁
```

Skip to content tants in the solutions from dsolve are symbols of the form `C1`, `C2`, `C3`, and so on.

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024